

Estimaciones Ágiles

¿Qué son las estimaciones? ¿Por qué se dice que las estimaciones son ágiles?

Predicción analítica e imparcial del tamaño o costo de un trabajo. Su fin principal es predecir la completitud y gestionar riesgos

Se las llama "**ágiles**" porque asumen la incertidumbre como una realidad ineludible y rechazan la falsa ilusión de control y precisión milimétrica de los métodos predictivos tradicionales. En lugar de ser un plan rígido, se adaptan a la realidad empírica del proyecto

- **Son relativas, no absolutas:** o estiman en horas o días, sino en tamaño, riesgo y complejidad mediante medidas relativas como los **Story Points**
- **Hechas por quienes ejecutan:** en ágil el equipo de desarrollo que hará el trabajo técnico es el único dueño de la estimación, realizándose de forma colaborativa
- **Rechazan el desperdicio (Lean):** Buscar precisión exacta al inicio es inútil porque los requerimientos inevitablemente van a cambiar. Es preferible destinar poco esfuerzo a estimar, aceptar la imperfección inicial y dejar que la métrica de "**Velocidad**" del equipo corrija y absorba las desviaciones con el paso de los Sprints.
- **Iterativas y "justo a tiempo":** No se intenta estimar todo el producto al principio, cuando el nivel de ignorancia sobre el sistema es máximo. Se estima continuamente a medida que aumenta la información, detallando las tareas recién al comienzo de cada **Sprint**.
- **Una estimación ágil es una predicción probabilística e imparcial, no una promesa inquebrantable si la estimación falló, no se extienden los plazos ni se recorta la calidad, simplemente se ajusta el alcance.**

La Dinámica de la Incertidumbre

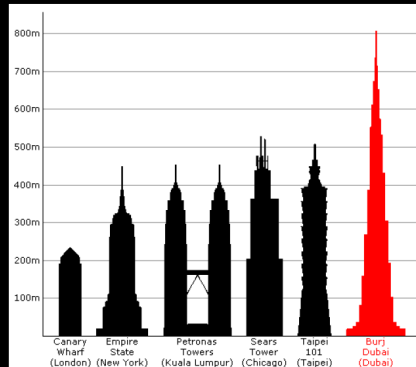
- Las estimaciones iniciales son inherentemente erróneas; el nivel de incertidumbre al comienzo del proyecto puede generar variaciones tempranas de hasta un 400%.
- La certidumbre se alcanza tomando decisiones de diseño, cerrando el alcance y construyendo software que funcione.
- Las estimaciones artificialmente bajas, impulsadas por la presión del negocio, crean una dinámica destructiva: fuerzan cronogramas irreales que obligan a recortar drásticamente la fase de análisis y diseño.

El tamaño se refiere al producto, el esfuerzo representa al trabajo necesario en horas por personas
Lineal → 1 persona = 1 tarea

Las US se estiman a partir de las SP, nosotros nos enfocamos en estimar el tamaño

Estimación Relativa

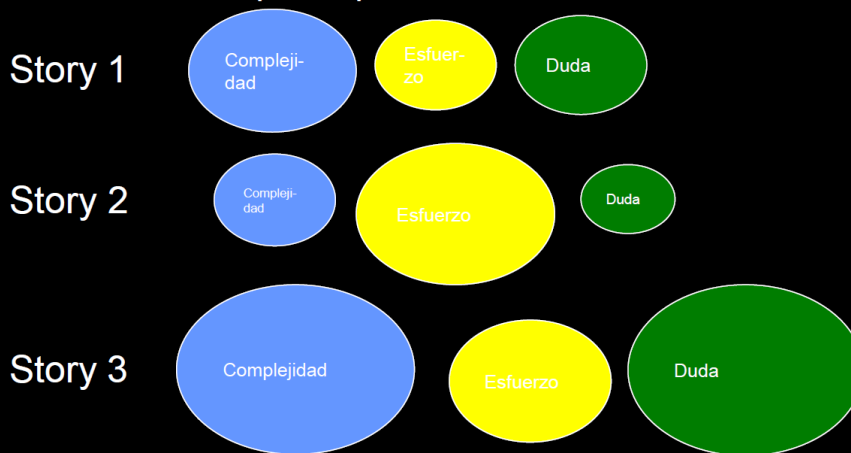
- Las personas no saben estimar en términos absolutos
- Somos buenos comparando cosas
- Comparar es generalmente más rápido.
- Se obtiene una mejor dinámica grupal y pensamiento de equipo más que individual
- Se emplea mejor el tiempo de análisis de las storys



Estimación Relativa y Story Points

- En el agilismo evitamos estimar en tiempo absoluto (horas lineales) porque los humanos somos ineficaces para ello, pero somos excelentes realizando comparaciones relativas entre elementos.
- El tamaño de una historia de usuario no equivale directamente al esfuerzo temporal; el esfuerzo variará dependiendo de la experiencia, conocimientos y habilidades de cada desarrollador.
- Un Story Point es una unidad de medida consolidada que representa tres factores integrados: la complejidad técnica, el esfuerzo necesario y el nivel de incertidumbre o duda sobre la tarea.
- Utilizamos la secuencia de Fibonacci (1, 2, 3, 5, 8, 13, etc.) para estas estimaciones, lo cual refleja matemáticamente que a mayor tamaño de la funcionalidad requerida, mayor será el margen de incertidumbre.

3 Stories que queremos estimar



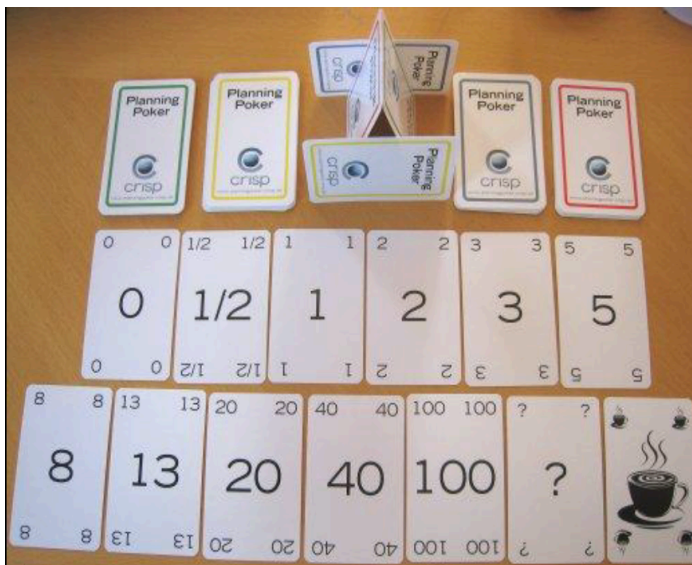
COMPLEJIDAD / ESFUERZO / DUDA

- **Complejidad:** Refleja la dificultad técnica o lógica inherente del requerimiento (ej. algoritmos complejos, validaciones cruzadas) y es una variable independiente de quién escriba el código.

- **Esfuerzo:** Indica el trabajo en horas-persona lineales necesarias para finalizar la tarea operativa. El esfuerzo no se puede sistematizar
- **Duda (Incertidumbre):** Representa la falta de información, ya sea técnica (uso de un framework nuevo) o de negocio (falta de definición clara por parte del Product Owner).

El tamaño desigual de los círculos demuestra visualmente que las historias crecen por motivos muy diferentes antes de unificarse en un único número de Fibonacci:

El tamaño de los círculos de Esfuerzo y Duda efectivamente depende de la experiencia previa y el conocimiento del negocio que tenga la persona o el equipo que lo va a desarrollar. Sin embargo, la complejidad es un factor intrínseco al problema a resolver y no se achica por el simple hecho de que un desarrollador sea senior.



El número representa el nivel de complejidad de una US, la carta con el valor cero (0) indica que el estimador quizás no tiene ninguna idea sobre el producto o la funcionalidad en ese punto específico.

¿se estiman a nivel de sprint? ¿por qué no a nivel proyecto?

No se estima a nivel proyecto porque al inicio la incertidumbre es máxima y los requerimientos son cambiantes, por lo que intentar detallar y predecir todo el alcance por adelantado resulta en estimaciones inherentemente erróneas y en un gran desperdicio de esfuerzo. Las estimaciones se realizan a nivel de Sprint

Planning Poker y Ecuación Ágil

- Garantiza que quienes ejecutarán el trabajo técnico sean precisamente quienes lo estimen, evitando imposiciones externas y fomentando el consenso del equipo.
- Los miembros del equipo revelan sus cartas de Fibonacci simultáneamente; si hay divergencias significativas, se debaten las justificaciones de los extremos para forzar conversaciones técnicas y alinear el entendimiento.
- La Velocidad (puntos completados y aceptados por iteración) actúa como una métrica empírica de progreso del equipo que permite absorber y corregir los errores de las estimaciones iniciales.

La velocidad nunca se estima, se mide empíricamente sumando los SP terminados al final de cada iteración, absorbiendo las desviaciones y corrigiendo los errores de las estimaciones iniciales. ej

US1= 3SP ❌

US2= 5 SP ✅ 5 SP primero se deduce la US canonica

US 3= 2SP ❌

La velocidad es la cantidad de US aprobadas por el PO, NO SE ESTIMAN. SE CALCULAN SUMANDO Y MULTIPLICANDO

- La regla de oro arquitectónica es estimar el tamaño, medir la velocidad empírica y, a partir de ese cálculo, derivar la duración total del proyecto. Las estimaciones analíticas nunca se negocian; lo único que se negocia es el compromiso, ajustando el alcance o las fechas de entrega.

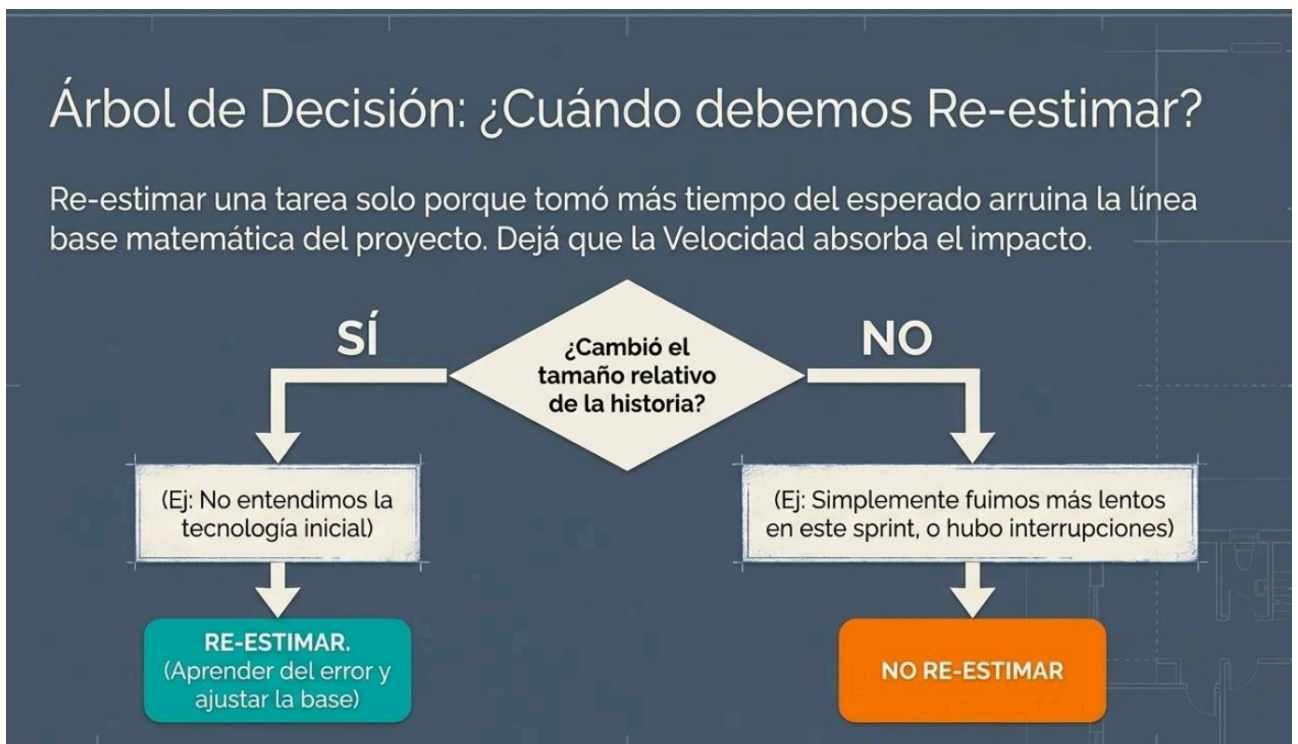
User story Canónica: ¿Qué es? ¿Por qué se deduce primero? ¿Qué pasa si se elige mal?

Es la historia base que se utiliza como punto de referencia para comparar y estimar el tamaño de todas las demás historias del proyecto. Se debe definir primero porque el proceso de estimación ágil se basa exclusivamente en medidas relativas y no absolutas; es indispensable establecer esta unidad de referencia inicial antes de poder evaluar el resto del trabajo.

El equipo suele elegir como canónica a aquella historia que presenta el menor grado de incertidumbre y se le asigna un valor inicial de 1 o 2 Story Points

Si la historia canónica se elige de forma errónea se compromete el "metro patrón" de todo el proceso de medición.

Dado que el resto de las historias se evalúan respondiendo a la pregunta de "cuán grande/pequeña, compleja o riesgosa es comparada con la canónica", una mala elección base distorsionará las proporciones relativas de todas las estimaciones subsiguientes.



La estimación **NO** se negocia. La estimación es diseño

¿Cuándo se debe o no re-estimar?

Cuándo Sí re-estimar

- Debés hacerlo únicamente cuando la opinión sobre el *tamaño relativo* de la Historia de Usuario (US) ha cambiado drásticamente.
- Esto ocurre si el equipo descubre nueva información que altera la complejidad o incertidumbre original, como comprender mal la tecnología requerida desde el inicio o notar grandes incoherencias al compararla con otras US recién agregadas al Product Backlog.
- En este escenario, re-estimar permite aprender del error técnico y ajustar la base de medición para mantener la coherencia en futuras estimaciones.

Cuándo NO re-estimar

- Nunca debés alterar el puntaje de una historia simplemente porque la tarea tomó más tiempo del esperado en finalizarse.
- Si hubo interrupciones, problemas externos o el equipo simplemente fue más lento durante el sprint, el tamaño intrínseco de la funcionalidad no se modificó; lo único que varió fue el esfuerzo aplicado.